

Week 11: Agent Runtime as a Server Workload

Tool Calls, Latency, and Tail Behavior

“An agent request is just a server request — only it expands into a tree of LLM and tool calls inside.”

§1 — What IS an Agent?

What did Cursor just do?

You: “*Fix the failing test in `auth.py`*”

Cursor (over ~30 seconds):

- Reads `auth.py`, then `test_auth.py`
- Runs `pytest tests/test_auth.py` — sees failure
- Edits `auth.py`
- Reruns `pytest` — now passes
- Reports back: “*Fixed by handling the empty-token case on line 42.*”

How many times did Cursor talk to the LLM?

How many times did it run something on your machine?

The Minimal Definition

An **agent** is an **LLM** + a set of **tools** + a **loop**.

Three components. We’ll define each in turn.

- **Tool** — a function with structured args and structured returns
- **LLM** — picks which tool to call next, and with what arguments
- **Loop** — Thought → Action → Observation → Thought → ...

That’s it. Everything else (planning, memory, RAG) is built on top.

Component 1: A Tool Is Just a Function

A tool is a regular function. It has:

- A **name**
- A **JSON schema** for arguments
- A **return value** (text, JSON, bytes, error)

```
def read_file(path: str) -> str:
    """Read a file from the local filesystem. Returns its contents."""
    with open(path) as f:
        return f.read()

def run_shell(cmd: str, timeout_sec: int = 10) -> dict:
    """Run a shell command. Returns {stdout, stderr, exit_code}."""
    ...
```

Key idea: these aren't "AI tools." They're **regular Python functions** the agent runtime knows how to call.

Component 2: The LLM as a Router

The LLM is given the tool schemas. On each turn it returns either:

1. "Call tool *X* with these arguments" (an **action**), or
2. "I'm done, here's the final answer"

A real OpenAI-style response:

```
{
  "role": "assistant",
  "tool_calls": [{
    "name": "read_file",
    "arguments": {"path": "/etc/hostname"}
  }]
}
```

The LLM is doing routing: given the user's goal and the conversation so far, which function should run next, with which args?

Component 3: The Loop

The pattern is called **ReAct** (Yao et al., 2022): Thought → Action → Observation → repeat. Each turn extends the conversation by one LLM-decided

1.00

Figure 1: 1.00

step.

“List the 3 largest files in /tmp” — Setup

User goal: “Show me the 3 largest files in /tmp.”

Tools available to the agent:

- `run_shell(cmd: str) → {stdout, stderr, exit_code}`
- `final_answer(text: str) → end loop`

Watch what happens turn by turn over the next 3 slides. Each turn is **one LLM call + at most one tool call**.

Turn 1: Plan and dispatch

LLM input: user goal + tool schemas **LLM output:**

```
{"tool_calls": [{  
  "name": "run_shell",  
  "arguments": {"cmd": "ls -lS /tmp | head -10"}  
}]}
```

Runtime executes: `ls -lS /tmp | head -10`

Tool returns:

```
total 1247632  
-rw----- 1 user user 524288000 Apr 20 11:14 cache.bin  
-rw-r--r-- 1 user user 314572800 Apr 19 09:02 dump.tar  
-rw-r--r-- 1 user user 10485760 Apr 18 14:33 build.log  
... (more lines)
```

Observation appended to context. Loop continues.

Turn 2: Refine — wrong tool result, ask again

LLM sees the `ls` output and notices: *the output includes the **total** block line plus 9 file entries, making it harder to parse cleanly. The user only wanted 3*

files.

LLM output:

```
{"tool_calls": [{  
  "name": "run_shell",  
  "arguments": {"cmd": "du -ah /tmp 2>/dev/null | sort -rh | head -3"}  
}]}
```

Tool returns:

```
500M    /tmp/cache.bin  
300M    /tmp/dump.tar  
10M     /tmp/build.log
```

Observation appended. This is the data the user actually wanted.

Turn 3: Final answer — exit the loop

LLM output:

```
{"tool_calls": [{  
  "name": "final_answer",  
  "arguments": {"text": "Top 3 files in /tmp:\n1. cache.bin (500M)\n2. dump.tar (300M)\n3. build.log (10M)"}  
}]}
```

Loop exits. User sees the answer.

Total cost:

- 3 LLM calls
 - 2 shell tool executions + 1 final-answer “tool”
 - ~5.5 s wall time: 3× LLM (~3 s) + 2× shell (~2 s) + orchestration (~0.5 s)
-

What an Agent Is NOT

Confusion	Reality
“Agent = chatbot”	Chatbots only emit text. Agents act on the world through tools.
“Agent = one big LLM call”	An agent is a loop . One task = many calls.
“Agent = magic black box”	Every loop step is observable , like a syscall trace.

Confusion	Reality
“Tool calls are different from function calls”	They’re regular function calls; the LLM just chose them.

Why an OS Course Cares

Three reasons agents belong in this course:

1. **They are the next user interface.** You’re already using Cursor, ChatGPT with code interpreter, Claude Desktop with MCP. Whatever replaces “shell + IDE” will be agent-shaped.
 2. **An agent is a server workload.** Tail latency, queueing, throughput, SLOs — all the Week 2 / Week 8 / Week 10 questions apply.
 3. **Failure modes are OS classics in new clothing.** Stuck loops look like livelock. Slow tools look like fsync stalls. Memory blow-ups from long context look like leaky processes.
-

Agent Frameworks

You’ll meet these in industry:

- **OpenAI Agents SDK** / **Anthropic claude-agent-sdk** — vendor-blessed, minimal
- **LangChain** / **LangGraph** — graph-based orchestration, batteries-included
- **LlamaIndex** / **CrewAI** / **AutoGen** / **pydantic-ai** — RAG-, multi-agent-, or type-safety-flavored

All of them implement the same three components you just saw: **LLM + tools + loop**. They differ in *defaults, abstractions, and lock-in*.

§2 — An Agent Request IS a Server Request

One Agent Task = Tree of Calls

A single user request expands into:

```
user query
  agent.task (root)
    llm.call #1    800 ms
    tool.call #1   200 ms
    llm.call #2   1200 ms
    tool.call #2   5000 ms  ← slow
    llm.call #3    600 ms
    tool.call #3   100 ms

                                total: 7900 ms
```

The Latency Identity

For a single-agent task:

$$T_{\text{agent}} = \sum_i T_{\text{LLM},i} + \sum_j T_{\text{tool},j} + T_{\text{orchestration}}$$

- $T_{\text{LLM},i}$ — each LLM call (depends on prompt size + output size)
- $T_{\text{tool},j}$ — each tool execution (depends on the tool)
- $T_{\text{orchestration}}$ — JSON parse, dispatch, IPC, your code

This is the budget you're managing.

Apply the Identity to Our /tmp Example

Term	What	Time (s)	%
$\sum T_{\text{LLM}}$	3 LLM calls	3.0	55%
$\sum T_{\text{tool}}$	2 shell commands	2.0	36%
$T_{\text{orchestration}}$	Python loop overhead	0.5	9%
Total		5.5	100%

The LLM calls dominate here. But swap one shell command for a slow HTTP API call (5 s) and tools flip to 70%+. **The bottleneck depends on the task — the identity tells you where to look.**

The Throughput Question

“A single server can handle N concurrent agent sessions.”

What’s N ?

It’s much harder to bound than for a single LLM service:

- Each session opens a long-lived loop with **unpredictable depth**
- Each session has **bursty** LLM and tool demand
- A “stuck” session holds resources until it times out

The SLO Question — Budgeting Across the Tree

End-to-end SLO: *“answer the user in under 5 seconds.”*

How do you spend that 5000 ms across a tree of unknown shape?

Component	Typical cost	Strategy
3× LLM calls @ 800 ms	2400 ms	Pick smaller model, shorten prompts
4× tool calls @ 400 ms avg	1600 ms	Parallelize, cache
Orchestration	~50 ms	Usually negligible
Slack	~950 ms	Buffer for tail behavior

If any single tool blows past its share — the whole task misses SLO.

Key Question

When an agent task is slow, where did the time go, and what can we do about it?

§3 — measurement (where did it go?) §4 — three classical levers (what can we do?) §5 — Lab F gives you a real agent to profile and fix.

§3 — Measurement: Anatomy of a Slow Agent

1.00

Figure 2: 1.00

Recap: Performance Methodology

“Measure first. Reason about mechanism. Only then optimize.”

Applies verbatim to agents:

1. **Instrument** — every LLM call, every tool call, every parse step
2. **Visualize** — a trace, not a flat log
3. **Diagnose** — which span dominates? what is its mechanism?
4. **Fix** — apply one of the three levers from §4
5. **Re-measure** — did the fix actually move the needle?

The biggest failure mode is **optimizing the wrong term**.

What to Measure (Per Step)

For each LLM call, record:

- input tokens, output tokens, model name
- TTFT (time to first token)
- total latency
- finish reason (stop, length, tool_calls, error)

For each tool call, record:

- tool name, argument hash
- start time, end time, exit status
- bytes in, bytes out

For the whole task:

- step count, total tokens, total \$ cost, total wall time
 - success / fail / timeout
-

Where Does an LLM Call’s Time Go?

Two regimes inside one LLM call:

- **Prefill** (TTFT) — scales with **input** token count $\times$ model FLOPs

- **Decode** — scales with **output** token count $\times$ model FLOPs

Where Does a Tool Call’s Time Go?

Order-of-magnitude estimates — measure your own in Lab F:

Tool kind	Typical latency	Dominated by
Local file read	1–5 ms	syscall + page cache
Shell command	10–100 ms	fork + exec
HTTP API call	50–500 ms	network RTT + remote service
Code-exec sandbox	500–5000 ms	container start + the code itself
Vector DB query	20–200 ms	embedding + ANN search

Tool latency varies by 4 orders of magnitude. A poorly chosen tool can dwarf every other cost in the loop.

From Logs to Traces

Logs tell you **what** happened. They don’t show **what caused what**. To find the slow span in an agent task, we need a richer record.

A **trace** is a tree of **spans**, where each span is one unit of work:

Field	Meaning
name	what this step is, e.g. <code>tool.read_file</code>
start, end	when it ran (so we know its duration)
parent_id	which span triggered me
attributes	typed key-values: model, tokens, args
status	ok / error

The **parent_id** is the magic ingredient — it’s what turns a flat stream of records into a causality tree:

```

invoke_agent           [root span, no parent]
  gen_ai.client.request [parent_id = invoke_agent]
    execute_tool: read_file [parent_id = gen_ai.client.request]

```

OpenTelemetry: One Standard for All Spans

OpenTelemetry (OTel) is a CNCF project — a vendor-neutral specification plus libraries for emitting traces, metrics, and logs. Two parts matter today:

1. **OTLP** — the wire protocol that ships spans from your code to a backend.
2. **Semantic conventions** — agreed-on attribute *names*. HTTP spans use `http.request.method`, DB spans use `db.system`, network spans use `net.peer.name`. The names are part of the standard.

Think of it as the **syscall ABI of observability**. POSIX standardizes `errno` values so any tool can read any program's errors; OTel standardizes span names so any backend can read any program's traces.

If your code emits OTel-compliant spans, you can swap backends — Jaeger, Tempo, Honeycomb, Datadog, LangSmith, Langfuse — without touching the instrumentation.

gen_ai.*: The Names for LLM and Agent Spans

For LLM/agent code, OTel defines a **gen_ai.*** family of attributes plus a few standard span names. Three span types cover today's lab:

Span name	Used for
<code>invoke_agent</code>	one full agent task (root span)
<code>gen_ai.client.request</code>	one LLM call
<code>execute_tool</code>	one tool call

A typical LLM-call span looks like:

```
Span: gen_ai.client.request
parent_id: <invoke_agent span id>
attributes:
  gen_ai.system           = "openai"
  gen_ai.request.model    = "gpt-4o-mini"
  gen_ai.usage.input_tokens = 1283
  gen_ai.usage.output_tokens = 47
  gen_ai.response.finish_reasons = ["tool_calls"]
duration_ms: 850
```

1.00

Figure 3: 1.00

Reading a Waterfall

We convert the causality tree to a 2D picture:

- **X axis** = time (ms)
- **Y axis** = depth in the tree (parent above, children indented below)
- Each span is a **bar** whose width is its duration

What you read off it:

- The **dominant span** — the widest bar (your first suspect)
- The **critical path** — the longest chain from root to leaf, which sets wall-clock time
- **Idle gaps** — white space between bars = orchestration overhead
- **Sibling layout** — staircase (serial) vs overlapping (parallel)

Worked Example: Diagnose This Trace

Think-pair-share: which span dominates? What’s the mechanism? What would you try first?

Four Common Patterns You’ll See

Most slow agent traces fall into one of:

#	Pattern	Visual signature
1	Prefill explosion — giant prompt dwarfs everything	One fat orange bar; rest tiny
2	Slow tool tail — one tool dominates	One fat blue bar; rest short
3	Sequential dependency — staircase of small steps	Staircase, no overlap
4	Repeated work — same tool, same args, again	Identical bars at regular intervals

These Are Old OS Problems in New Clothes

Agent pattern	Classical OS analog
Prefill explosion	Long syscall argument processing
Slow tool tail	Slow device IO (Week 8: fsync)
Sequential dependency	Serial syscalls without <code>io_uring</code> batching
Repeated work	Missing page cache / memoization
Stuck loop	Livelock / fork bomb

How Real Teams Ship This

You won't hand-write a tracer in production. The stack has three layers:

Layer	Role	Common choices
Instrument	Auto-emit <code>gen_ai.*</code> spans from LLM/tool calls	OpenLLMetry, OpenInference, OpenAI Agents SDK
Transport	Ship spans over OTLP	OpenTelemetry Collector
Backend	Store + render the waterfall	LangSmith, Langfuse, Arize Phoenix

The ~30 LoC of `tracer.py` you'll write in lab collapses to one line in production:

```
from openinference.instrumentation.openai import OpenAIInstrumentor
OpenAIInstrumentor().instrument() # every LLM call + a gen_ai.client.request span
```

The other half of agent debugging — *eval-driven regression testing* (golden datasets + LLM-as-judge: Braintrust, LangSmith Evals, Promptfoo). Same trace data, different question: “*did my change break behavior?*” Out of scope today.

Before We Jump to Solutions...

In the trace waterfall we just analyzed (slide “Diagnose This Trace”):

What was your diagnosis? Which span dominated, and why?

We'll collect 1–2 answers before §4.

After Measurement, Three Levers

For every diagnosis you'll reach for one of three optimizations:

- **Concurrency** — run independent tool calls in parallel (§4a)
- **Caching** — avoid recomputing identical work (§4b)
- **Timeout + fallback** — bound the worst case (§4c)

These are the same three levers Linux uses for IO, the same three your browser uses for HTTP, the same three you'll use here.

§4 — Three OS Levers Applied to Agents

Concurrency, caching, and bounded waits

Warm-up: Revisit the /tmp Quick Check

Could we have parallelized the two `run_shell` calls in our `/tmp` example?

No. Turn 2's `du` command depended on Turn 1's `ls` output — the LLM read Turn 1's observation before deciding Turn 2's action.

The dependency goes through the LLM. Two tool calls can run in parallel only if they appear in the **same** LLM turn and don't depend on each other.

This is the DAG insight we'll formalize next.

§4a — Concurrency: Sequential vs Parallel

Default: tool calls run **one at a time**.

T1
T2
T3

(each waits for the previous)

But OpenAI / Anthropic both support **parallel function calling**: the LLM can emit multiple tool calls in **one** turn.

T1
T2

T3

(dispatched concurrently)

If the calls are independent, you save $T_1 + T_2 + T_3 - \max(T_1, T_2, T_3)$.

The DAG View — Only Independent Calls Parallelize

`search_web("X")` `read_calendar()` ← can run in parallel

LLM

`draft_email()` ← must wait for both

`send_email()` ← must wait for draft

The shape determines the speedup. Wide DAG → big win. Linear chain → no win.

Amdahl Is Still the Boss

$$\text{Speedup}_{\max} = \frac{1}{(1-p) + p/N}$$

If LLM time = 60% of total and you parallelize **all** tool time:

- $p = 0.4, N \rightarrow \infty \rightarrow \text{Speedup}_{\max} = 1.67\times$
- Best case: 8 s task → 4.8 s task

You cannot parallelize the LLM serial chain. The LLM has to see each turn's observations before deciding the next turn.

§4b — Caching at Three Layers

Layer	What's cached	Who manages
LLM prompt cache	KV cache for repeated prompt prefixes	Provider (50–90% discount)
Tool result cache	<code>tool_name + args_hash → result</code>	Your runtime

Layer	What's cached	Who manages
Embedding cache	text_hash → embedding vector	Your runtime (RAG)

All three follow the **same** principle as the OS page cache: *exploit locality of reference*.

When NOT to Cache

Caching is **wrong** when:

- Tool has **side effects** — write_file, send_email, run_payment
- Result is **time-sensitive** — current price, current weather, “now”
- Result depends on **hidden state** — random seed, user session
- **Privacy** boundary — different users must not share results

Rule of thumb: cache pure, side-effect-free reads. Never cache writes.

§4c — One Slow Tool Kills p99

Suppose 4 tools, all p50 = 50 ms. Three are well-behaved (p99 = 60 ms). One has a long tail (p99 = 10 s).

Under load, the slow tool's tail **dominates** end-to-end p99 latency — even though it's only 1 of 4 tools and rarely slow.

Defense: Timeout + LLM-Driven Fallback

Give every tool a **deadline**. On timeout, don't error — return a structured “timed out” observation and let the LLM decide.

```
try:
    result = await asyncio.wait_for(tool(**args), timeout=2.0)
except asyncio.TimeoutError:
    result = {"error": "timeout", "elapsed_s": 2.0,
             "hint": "tool may be unavailable; try alternative"}
```

The LLM is your fallback policy. It can retry with smaller args, pick a different tool, or report partial results to the user.

This is **agent-specific** — no traditional system has the recovery policy live in a language model.

Putting It Together — Each Lever Attacks a Different Problem

Same task, variants measured independently from baseline:

Variant	p50	p99	What it fixes
Baseline	8.0 s	22 s	—
+ parallel calls only	5.5 s	20 s	p50 on wide-DAG tasks
+ cache only	6.0 s	14 s	repeated-tool overhead
+ timeout only	6.0 s	9 s	p99 tail
All three combined	3.2 s	5.5 s	everything

Parallelism drops p50. Cache saves repeated work. Timeout kills the tail. Each lever attacks a different term in the latency identity.

§5 — Lab Preview

Lab F Preview: Profile and Optimize a ReAct Agent

You'll be given a **baseline ReAct agent** (~150 LoC Python, mock LLM, local tools — no API keys, no network).

Four parts (90 min):

1. **Profile** — instrument with OTel-style spans, build a waterfall
2. **Parallelize** — convert sequential tool calls to parallel where the DAG allows; measure the speedup
3. **Tame the tail** — inject a slow tool, add timeout + fallback, measure p50 / p99 before vs after
4. **Cache** — add a tool-result cache, measure repeat-task speedup

Deliverable: a perf report with before/after numbers for each lever and a 1-paragraph diagnosis per part.

Questions?

Reading for next week: Week 12 final-project demo day — bring your projects, your reproduction scripts, and your trace files.